

Bootcamp

Data Science ✓

Machine Learning ✓

Statistics +

Python +

Artificial Intelligence ✓

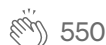
Maximum Likelihood Estimation Explained Simply



Kamrun Nahar

Follow

20 min read · Jun 28, 2026



550



6



2



The Scariest Formula in Statistics Is Actually a Coin Trick

A friend pushes a coin across the table. “This one’s rigged,” he says. “I can feel it.”

I pick it up. It looks normal. No little label, no blinking light, nothing that screams villain. So I do the dumbest scientific thing a person can do. I start flipping it.

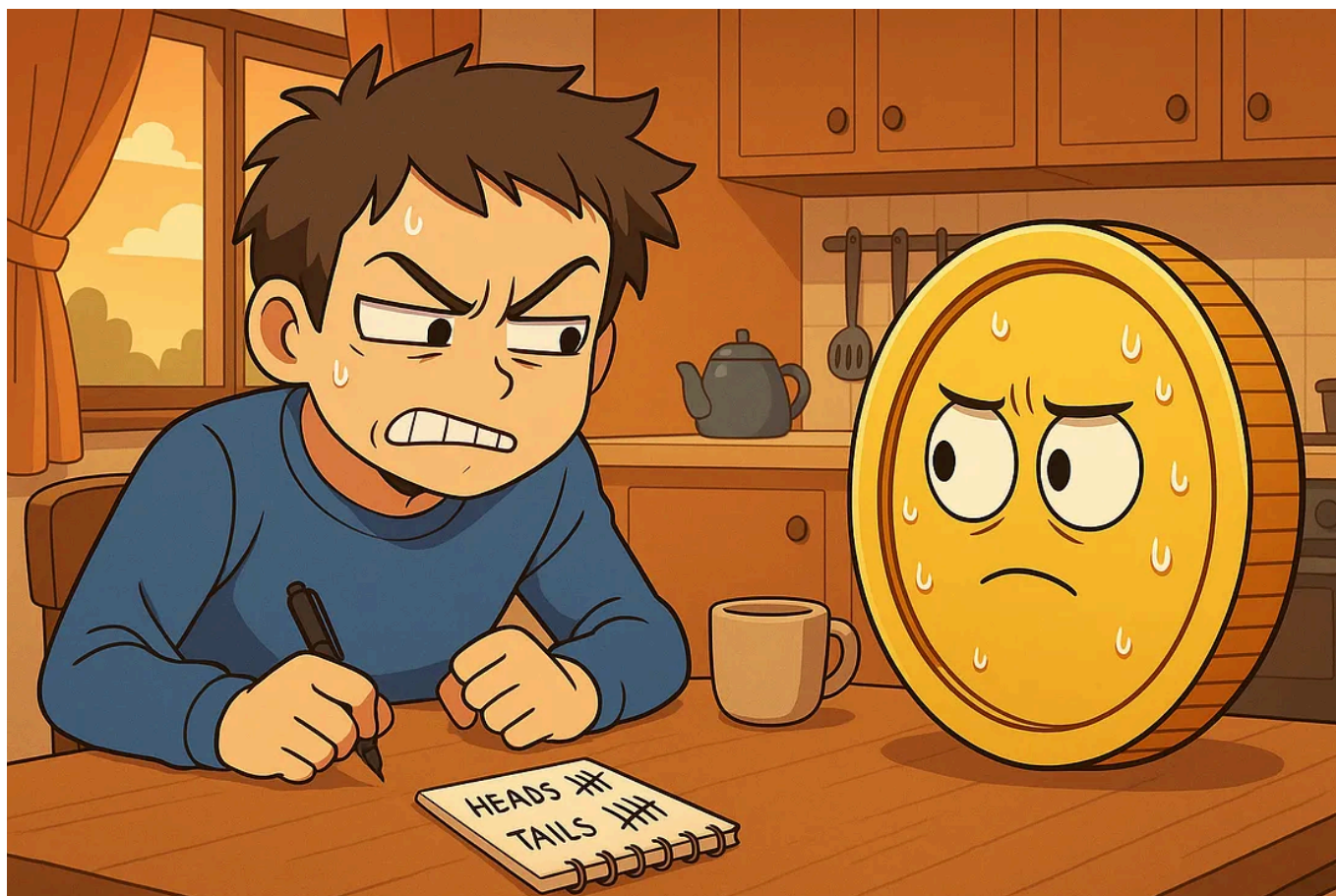
Heads. Heads. Tails. Heads. Heads. Tails. Heads. Heads. Tails.

Ten flips. Seven heads. And before I even finished counting, a number had already walked into my head and sat down. Seventy percent. I didn't compute anything. My brain just looked at seven out of ten and shrugged out a guess, the way it guesses how full a bus is before the doors open.

Here's the thing nobody mentions. That lazy little shrug is one of the most important ideas in all of statistics. It has a name that sounds like a parking fine. It runs your spam filter. It finishes your text messages. It sits inside the model your bank uses to decide if that 2 AM charge was really you.

It's called maximum likelihood estimation.

And the whole point of this article is simple. By the end, you'll get it. Not "nod along" get it. Actually get it, well enough to annoy someone at dinner with it. My dog is barking at the mailman as I write this, so let's move quickly.



The coin swears it is innocent. The tally marks say otherwise.

The whole idea, in one breath

Let me hand you the punchline now. I respect your time and I know your attention span has been through a lot today.

Maximum likelihood estimation means picking the explanation that makes your data look the least surprising.

That's it. Frame that. Everything else in this article is just us walking around that sentence, poking it, and watching it hold up.

Read it once more, slower. You have some data. Seven heads, three tails. And you have a pile of possible explanations for it. Maybe the coin is fair. Maybe it favors heads seven times in ten. Maybe it is glued to land heads forever. Maximum likelihood tries each explanation on like a jacket and asks one question. Under this explanation, how ordinary does my data look.

A fair coin can give you seven heads in ten. Sure. It happens. But a coin that lands heads 70 percent of the time makes that exact result feel completely boring, like a Tuesday. So your gut leans toward the boring explanation. Smart gut.

You have been doing this your entire life. We are just going to put a tie on it and give it a job title.

The easiest way to think about it, forever

Here is my one metaphor for this. Steal it.

You are a detective who always shows up after the crime.

You never see what happened. You only walk in later and stare at the mess. The broken lamp. The muddy boot print. The dog that did not bark. And from that mess, you reason backward to the most believable story. You do not get to pick the crime. You only pick the suspect who would most obviously leave exactly this mess.

That is maximum likelihood. The coin already flipped. The data already happened. You walked in late, like always, and now you point at the

explanation that fits the evidence best.

Keep that detective in your head. We will come back to him.

First, a word that ruins everything if you skip it

There are two words that look like twins and are not. Probability and likelihood.

In normal life they mean the same thing. “What’s the probability it rains.” “What’s the likelihood she texts back.” Nobody cares about the difference at a bus stop.

In statistics they are different jobs. And the whole magic lives in that gap. So let me make it stick.

Probability runs the movie forward. You know the rule, and you predict the data. You know the coin is fair, so you ask how many heads you’ll probably see. You are the director. You set the rules and watch the scene play.

Likelihood runs the movie backward. The data already happened, on tape, done. Now you ask which rule makes that footage make sense. You are not the director anymore. You are our detective, late again, staring at seven heads and asking what kind of coin does this.

Probability goes from cause to effect. Likelihood crawls from effect back to cause. Almost all of real statistics is the crawling kind, because we never

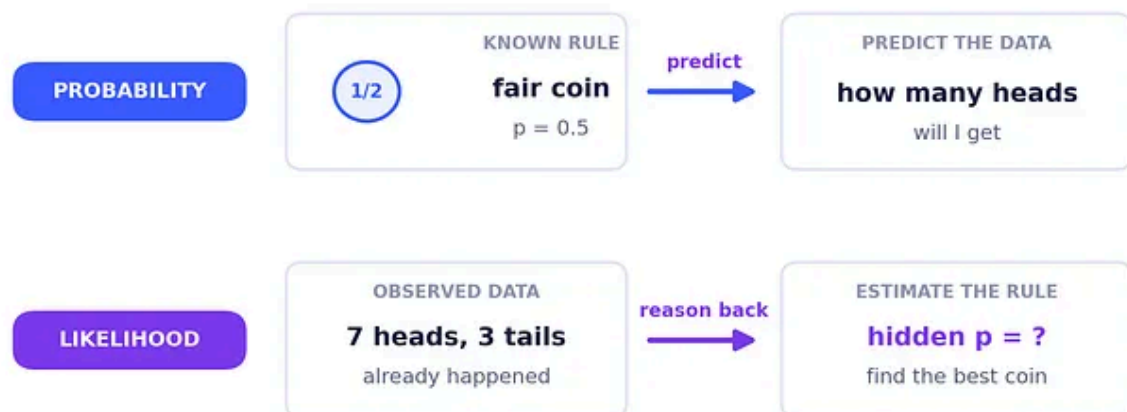
actually know the rules of the world. The world does not email us its settings. It just hands us a mess and walks off.

Here it is as a tiny table, because some things click better in a grid.

The question	Probability	Likelihood
What is fixed	the rule (the coin)	the data (the flips)
What is unknown	the data	the rule
Direction	rule $\rightarrow$ data	data $\rightarrow$ rule
Your role	the director	the detective

Probability vs Likelihood

Same coin. Opposite directions of thinking.



Probability runs the movie forward. Likelihood rewinds it from the evidence.

Same coin, same numbers, two completely different jobs. One predicts, one reasons backward.



He never sees the crime. He just shows up and judges the coin.

Okay, the coin. Let us actually do the thing.

Enough talking about it. Time to run it.

Picture the coin with one hidden dial on its side. The dial sets the chance of heads. We will call that setting p . If p is 0.5 the coin is fair. If p is 0.7 it lands heads seven times in ten. If p is 0.95 it is basically a heads vending machine that coughs up a tails by accident.

We cannot see the dial. That is the entire problem. The dial is hidden and the coin is not telling. This hidden dial has a fancy name in statistics. We call it a

parameter. Do not flinch at the word. A parameter is just a knob the universe set and then hid from you. That is all it ever means.

So let us go hunting for the knob.

Start with a guess. Say the dial sits at 0.5, a fair coin. How surprised should we be to see seven heads and three tails?

Each head on a fair coin has a 0.5 chance. Each tail, also 0.5. The flips do not talk to each other. The coin has no memory, no matter what your uncle says at the card table. So to get the chance of the whole run, we multiply the chances together. Ten flips, each one a 0.5.

```
chance of our data if p = 0.5
= 0.5 x 0.5 x 0.5 x 0.5 x 0.5 x 0.5 x 0.5 x 0.5 x 0.5 x 0.5
= 0.5 to the power 10
= about 0.000977
```

That tiny number is the likelihood of a fair coin given our data. It looks scary and small. Ignore the size. A likelihood number alone means nothing, the same way one price tag means nothing until you see a second shop.

So let us open the second shop. Try the dial at 0.7.

Now each head has a 0.7 chance and each tail has a 0.3 chance, because tail is just “not head” and the two have to add up to one. Seven heads, three tails.


```
chance of our data if p = 0.7
= 0.7 x 0.7 x 0.7 x 0.7 x 0.7 x 0.7 x 0.7 x 0.3 x 0.3 x 0.3
= (0.7 to the power 7) x (0.3 to the power 3)
= about 0.00222
```

Look at the two numbers. 0.00222 beats 0.000977. More than twice as big.

Sit with what that means. Our seven heads feel more than twice as at home in a world where the coin favors heads than in a world where it is fair. The 70 percent coin is less shocked by what we saw. Our detective leans toward it. Slowly. Squinting.



Same evidence. One coin owns it. One coin is sweating.

Why stop at two guesses

We tried 0.5 and 0.7. But the dial is smooth. It can sit at 0.5, or 0.63, or 0.701, or anywhere between zero and one. So let us be greedy. Let us test all of them.

For every possible dial setting, we ask the same question. If the dial were here, how unsurprising is my seven out of ten. We get a number for each setting. Then we plot it. Dial setting along the bottom. Likelihood going up.

When you do that for every value, you do not get a random scribble. You get a smooth hill.



Every possible coin, ranked by how well it explains our data. The hill peaks at 0.7 and looks pretty smug about it.

And here is the satisfying part. The hill has exactly one top. It sits right above 0.7.

That top is the dial setting that makes our data the least surprising of all. It is the single best explanation we have. It is the maximum of the likelihood. Which is, and I cannot believe how on the nose this name is, the maximum likelihood estimate.

The peak sits at 0.7. Which is seven divided by ten. Which is the exact number my lazy brain blurted out on flip number ten, before I knew any of this existed.

That is not luck. That is the whole machine of statistics quietly agreeing with your gut and handing you a receipt. For a coin, the maximum likelihood estimate is always just heads divided by flips. The math sweats through hills and powers and multiplication and lands precisely where common sense was already standing, tapping its foot, checking the time.

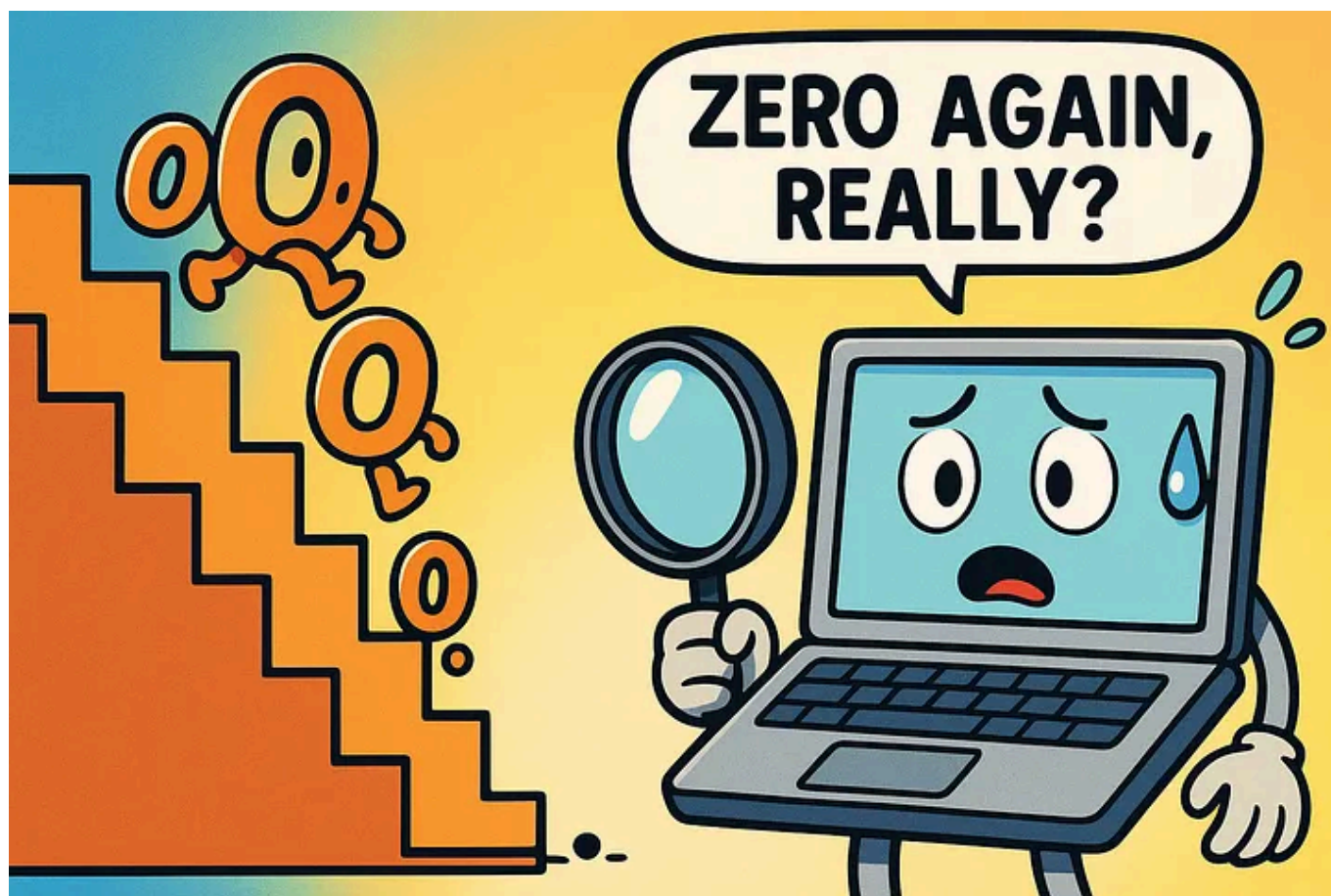
Why this matters. When your instinct is good, this method backs you up with proof. And when your instinct is useless, which is most of real life, the method keeps working anyway. That is the part you are really buying.

A problem we are about to smack into

Multiplying ten probabilities gave us 0.00222. Already uncomfortably tiny. Now picture not ten flips but ten thousand data points, which is a slow afternoon for any real dataset. You would be multiplying ten thousand numbers, each smaller than one.

The result gets so close to zero that your computer just gives up, rounds it to flat zero, and moves on with its day. This is a real bug with a real name. Numerical underflow. It has quietly broken more student projects than a dead laptop battery.

Multiply enough tiny things and you get one useless speck. We need a trick. Lucky for us, it is a famous one, and it is softer than it sounds.



Multiply enough tiny numbers and your laptop just rounds the whole thing to nope.

The log trick, the only fancy move in here, and it is gentle

The move is this. Instead of the likelihood, we work with the logarithm of the likelihood. We call it the log likelihood, and statisticians cannot get enough of it.

If the word logarithm just tightened your shoulders, drop them back down. You need exactly one fact about logs today, and this is the whole reason we bother.

| A logarithm turns multiplication into addition.

That is the trick. The log of a times b times c equals $\log a$ plus $\log b$ plus $\log c$. Multiplication, the thing that was crushing our numbers into dust, becomes addition, which is calm and never panics and never underflows.

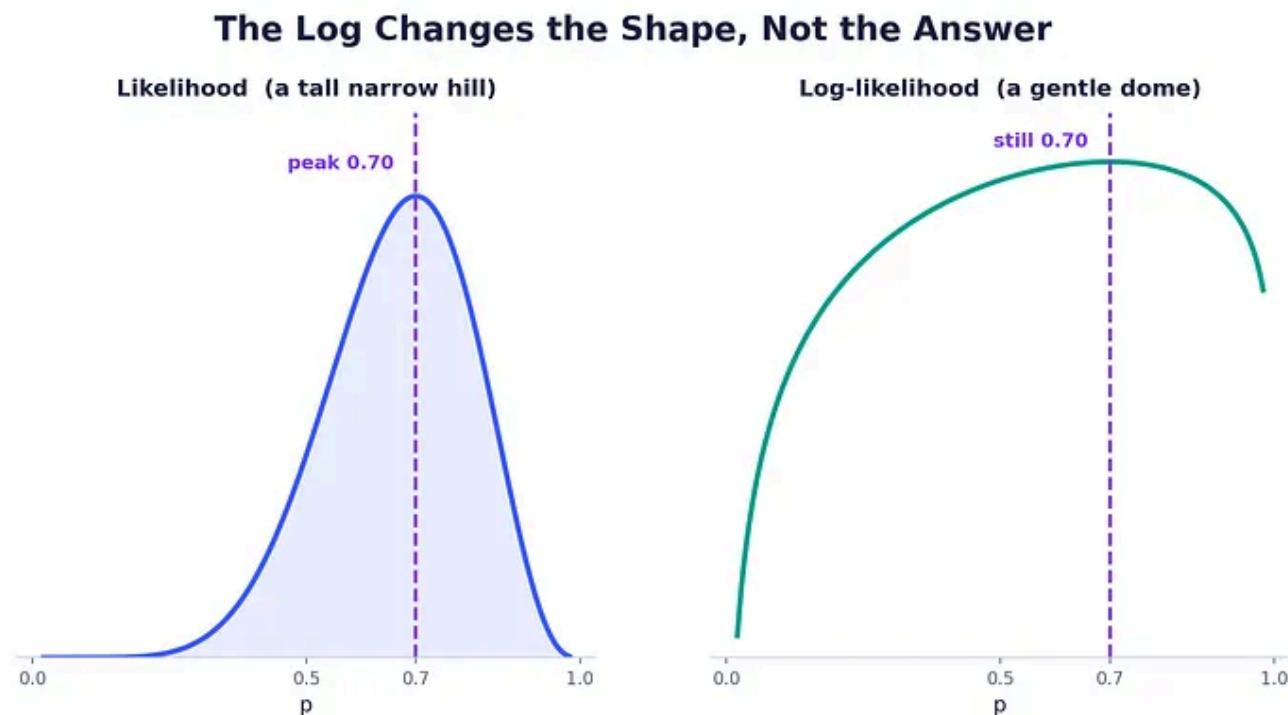
So instead of multiplying ten vanishing probabilities, we take the log of each one, which turns them into normal sized negative numbers, and we just add them up. A nasty product becomes a tidy sum. Your laptop relaxes.

Now the smart objection, which is you, arms crossed. Doesn't changing the numbers change the answer. If I take the log of the whole hill, isn't that a different hill with a different top.

Great instinct. Here is why it is safe.

A logarithm keeps order. That is the magic word, monotonic, but forget the word and keep the meaning. If one number is bigger than another, its log is also bigger. Logs never swap which value is on top. They can squish the hill, flatten it, lower it, turn a sharp peak into a soft dome, but they cannot slide the peak sideways. The highest point stays over the exact same dial setting.

So we get the whole benefit, addition instead of multiplication, no vanishing, and we pay nothing. The peak stays parked at 0.7. We just take a comfier road to reach it.



The log squashes the hill into a friendlier dome, but the peak refuses to move. Same answer, calmer math.

Why this matters. Every real stats library and every line of model training code you will ever read maximizes the log likelihood, not the raw likelihood. Not because anyone loves logs. Because it is the same hill with better footing. When you see “log likelihood” in the wild, just hear “the likelihood, made safe for computers.”

A short true story, because this did not fall from the sky

Nobody woke up with this fully formed. It was dragged into the world over about a decade by one famously difficult genius.

In the early 1920s a young British statistician named Ronald Fisher was chewing on a question that had bugged people for a century. When your data is messy and you want to estimate something, which estimate do you actually trust. There were rival recipes and a lot of hand waving and a heavy reliance on an older method that asked you to guess things before you had any right to.

Fisher, who had roughly zero patience for other people's methods, basically said stop guessing in advance. Let the data talk. Pick the setting that makes the data you actually saw the most probable. He sketched the numerical idea around 1912 as a student, named the concept likelihood around 1921, and laid out the full theory in a landmark 1922 paper.

It changed how people thought. Instead of "what did I believe before the data showed up," Fisher asked "what does the data, on its own, point at hardest." Maximum likelihood became a load bearing wall of modern statistics, and Fisher went on to invent an unfair amount of the rest of the field too.

I bring him up so you feel the weight of the thing in your hands. The coin on your table connects by a straight line to a hundred years of people trying to reason honestly from half the story. Not bad for ten flips and a barking dog.

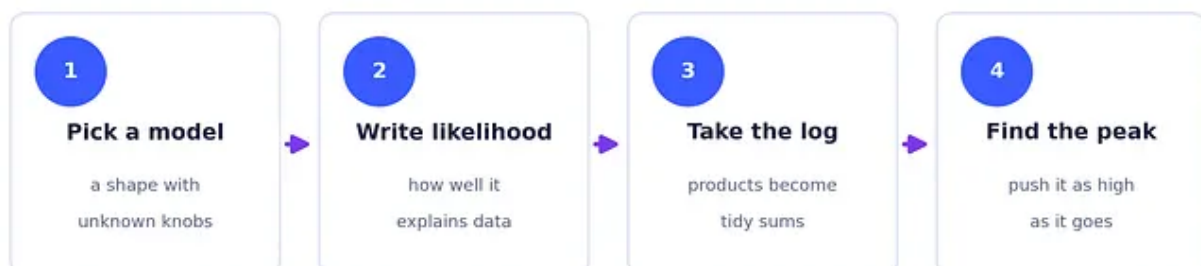


Ronald Fisher, the man who told everyone to stop guessing before the data arrived.

The recipe you can reuse on literally anything

We did the coin by feel. Let us write the actual recipe, because the beautiful part is that it never changes. A coin, the height of strangers, customer cancellations, the brightness of a far away star. Same four steps every single time.

The Maximum Likelihood Recipe



The four step recipe behind every maximum likelihood estimate, from a coin to a neural network. Memorize the shape, not the symbols.

Step one. Pick a model with a knob, or a few. Decide the rough shape of the world and leave the exact setting blank. For a coin the shape is “heads happens with chance p ,” and p is the blank. For heights the shape might be “heights follow a bell curve with some center and some spread,” and now you have two blanks. The blanks are your parameters.

Step two. Write the likelihood of your real data. For each data point ask how probable it is under a given knob setting, then combine them. Since data points usually do not affect each other, you multiply. You end up with one formula that scores how unsurprising your whole dataset is for any setting.

Step three. Take the log. Turn that product into a sum for all the sanity reasons we covered. Now you hold the log likelihood. Same peak, friendlier road.

Step four. Find the knob settings that push the log likelihood as high as it goes. Find the top of the hill. For easy cases a little calculus hands you a clean formula, often something almost rude in how simple it is, like “just take the average.” For hard cases you hand the hill to a computer and say climb, and it climbs.

Those settings at the summit are your maximum likelihood estimates. Done. The recipe scales from a coin you do not trust all the way up to a model that writes poetry, and the bones never change.

Let us watch it work in Python

Enough hand waving. Let us make a computer find the peak so you can see there is no card up my sleeve. We will use the same seven heads coin. The Enter key on my laptop sticks a little, so forgive me if I sound impatient.

```
import numpy as np
# I import numpy because doing this by hand at 1 AM should be a crime.
flips = np.array([1, 1, 0, 1, 1, 0, 1, 1, 1, 0])
# 1 is heads, 0 is tails. This is the actual evidence. Ten real flips.
heads = flips.sum()
# sum adds up all the ones, which just counts the heads. Here it is 7.
total = flips.size
# size counts how many flips happened in total. Here it is 10.
def log_likelihood(p):
    # p is a guess for the coin's hidden chance of heads, from 0 to 1.
    # np.log is the natural logarithm, our trick against vanishing numbers.
    # each head adds log(p), each tail adds log(1 minus p). We add, not multiply
    return heads * np.log(p) + (total - heads) * np.log(1 - p)
candidate_p = np.linspace(0.001, 0.999, 1000)
# linspace makes 1000 evenly spaced guesses from almost 0 to almost 1.
# we skip exactly 0 and 1 because log(0) blows up and ruins everything.
scores = log_likelihood(candidate_p)
# this scores all 1000 guesses at once. numpy does the whole batch in one shot.
best_p = candidate_p[np.argmax(scores)]
# argmax finds where the highest score sits. We grab the p standing there.
print(round(best_p, 3))
# prints 0.7. The computer just agreed with your gut. Try to act surprised.
```

Run it and you get 0.7. The computer climbed the hill by brute force, tested a thousand coins, and planted a flag at 0.7. No formula. Just try everything and keep the best.

Why this matters. This is the rawest possible version of the idea. Score every option, keep the winner. Every fancier method is just a smarter way to find

the same peak without checking a thousand points by hand.

Now let me show you the grown up way, the one that scales when you cannot just try a thousand values.

```
import numpy as np
from scipy.optimize import minimize_scalar
# minimize_scalar climbs the hill for us when there is no neat formula around.
flips = np.array([1, 1, 0, 1, 1, 0, 1, 1, 1, 0])
heads, total = flips.sum(), flips.size
def neg_log_likelihood(p):
    # we flip the sign because the optimizer only knows how to walk downhill.
    # minimizing the negative is the same as maximizing the original. Sneaky but
    return -(heads * np.log(p) + (total - heads) * np.log(1 - p))
result = minimize_scalar(neg_log_likelihood, bounds=(0.001, 0.999), method="boun
# bounds keep the search inside legal coin land, between 0 and 1, never outside.
print(round(result.x, 3))
# prints 0.7 again. Brute force, calculus, optimizer. They all shake hands.
```

Why this matters. Notice the trick. We did not maximize anything. We flipped the sign and minimized the negative log likelihood instead. Hold onto that. It is the secret handshake between statistics and machine learning, and we will cash it in soon.



The optimizer does the climbing. You do the socks and the snacks.

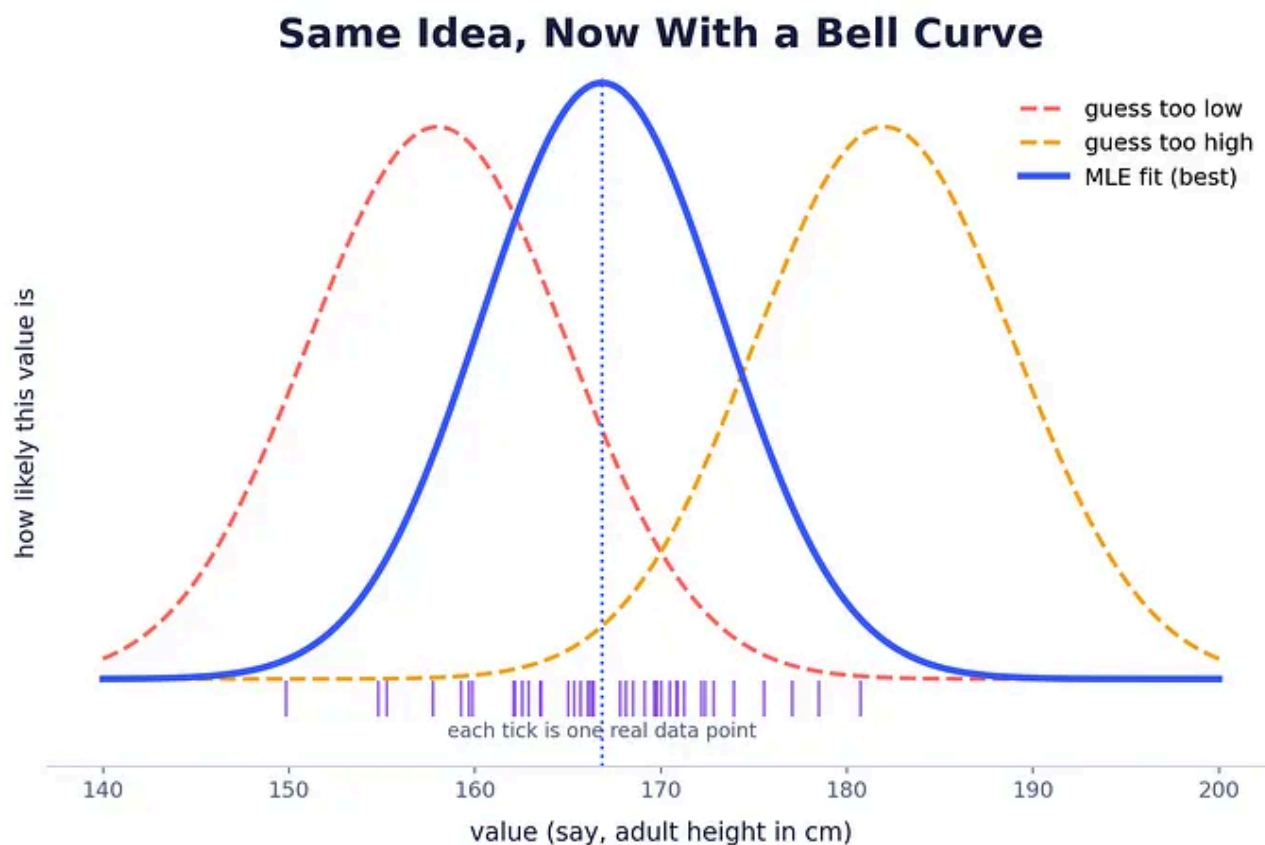
Same idea, now with a bell curve

The coin is great but it is binary, just heads or tails. Real data is often a smear of numbers. Heights. Weights. Wait times for the bus. So let me show the exact same idea on something continuous, because this is where it goes from cute to genuinely useful.

Say I measure ten people's heights in centimeters. I believe heights roughly follow a bell curve, the famous hump shape. A bell curve has two knobs. The

center, which is where the hump sits, and the spread, which is how wide and lazy the hump is.

Maximum likelihood asks the same question it always asks. Which bell curve makes my ten heights look the least surprising. Slide the center too far left and half my data sits way out in the unlikely tail, which is bad. Slide it too far right, same problem on the other side. There is one center and one spread that hug the data best.



Three guesses at the bell curve, one clear winner. The MLE fit is the curve that hugs the real data points instead of arguing with them.

And here is the punchline that should make you grin. When you grind through the math for a bell curve, the best center turns out to be the plain old average of your data. The best spread turns out to be the standard

deviation. The scary continuous case collapses into “take the average,” which you learned in school and probably resented at the time.

```
import numpy as np
data = np.array([171, 165, 180, 158, 169, 174, 162, 177, 168, 173])
# pretend these are ten people's heights in centimeters. Real numbers, not coins
mu_hat = data.mean()
# the MLE for the center of a bell curve is literally just the average. That is
sigma_hat = data.std()
# the MLE for the spread is the standard deviation. numpy hands it over for free
print(round(mu_hat, 2), round(sigma_hat, 2))
# this is the single most likely bell curve to have produced your ten heights.
```

Why this matters. Every time you have ever taken an average, you were secretly doing maximum likelihood estimation under a bell curve assumption. You were doing graduate statistics in middle school and nobody told you. The little hat on mu, by the way, just means “our estimate of,” not the true hidden value. People write it that way to stay humble.

A tiny peek at the calculus, no pain I promise

You can skip this box and lose nothing. But if you ever wondered how the computer “finds the top,” here is the honest one paragraph version.

The top of a smooth hill is flat. Think about standing on a hilltop. For one step in any direction, you are not going up anymore. That flatness is the whole secret. In math, the slope of the curve is called the derivative, and at the very top the slope is zero. So “find the peak” becomes “find where the slope is zero.” For the coin, you write the slope of the log likelihood, set it

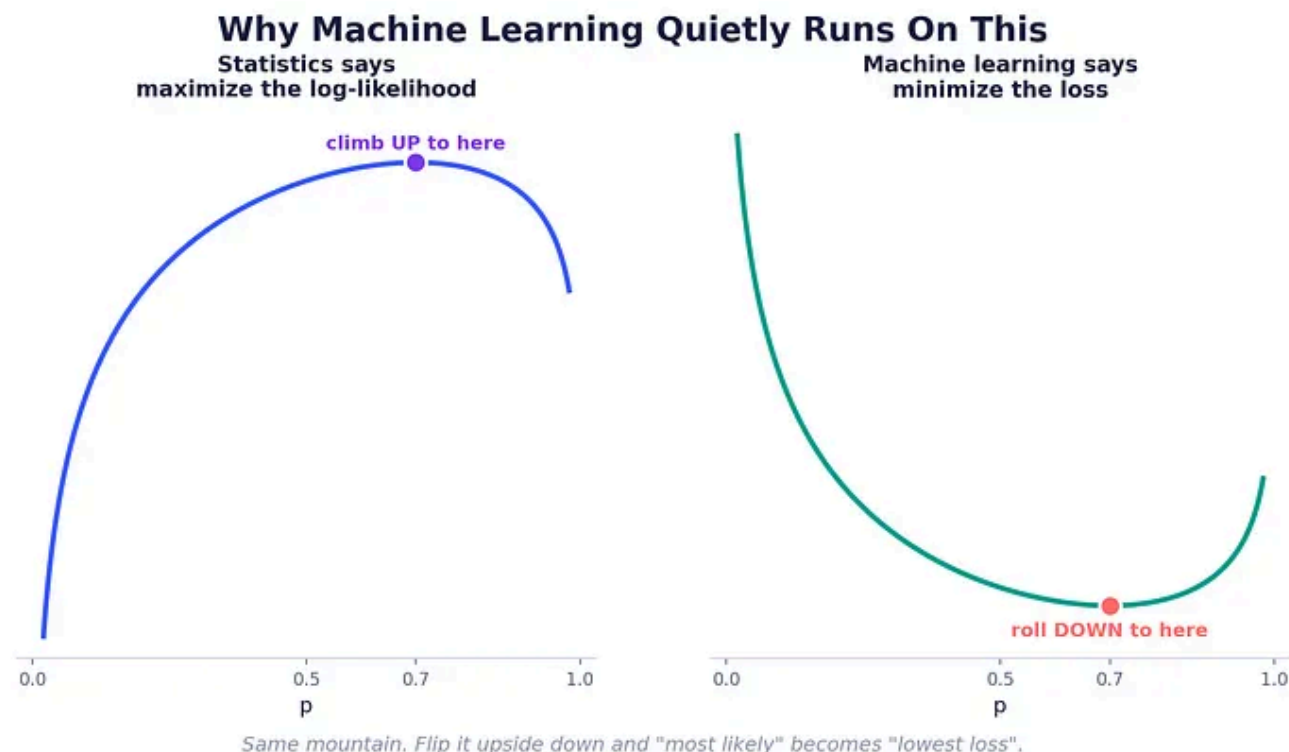
equal to zero, do three lines of tidy algebra, and out pops heads divided by total. The calculus is not the idea. The calculus is just the elevator to the top of a hill you already understood.

Why this matters. Whenever someone says they “took the derivative and set it to zero,” do not panic. They are just standing on the hilltop checking that the ground went flat. Same peak, formal shoes.

Why machine learning quietly runs on all of this

Here is where I pay off the promise from the top. You did not read this far to win a coin bet. You read it because this idea is the engine under a wild amount of the tech in your pocket. Once you know its shape, you start seeing it everywhere, like noticing a word for the first time and then hearing it four times that week.

Remember the sign flip from the Python. Maximizing the log likelihood is the same as minimizing the negative log likelihood. Statistics likes to climb up to “most likely.” Machine learning likes to roll down to “lowest loss.” Those are the same mountain, one of them just turned upside down.



Flip the hill upside down and most likely becomes lowest loss. Statistics and machine learning were climbing the same mountain the whole time.

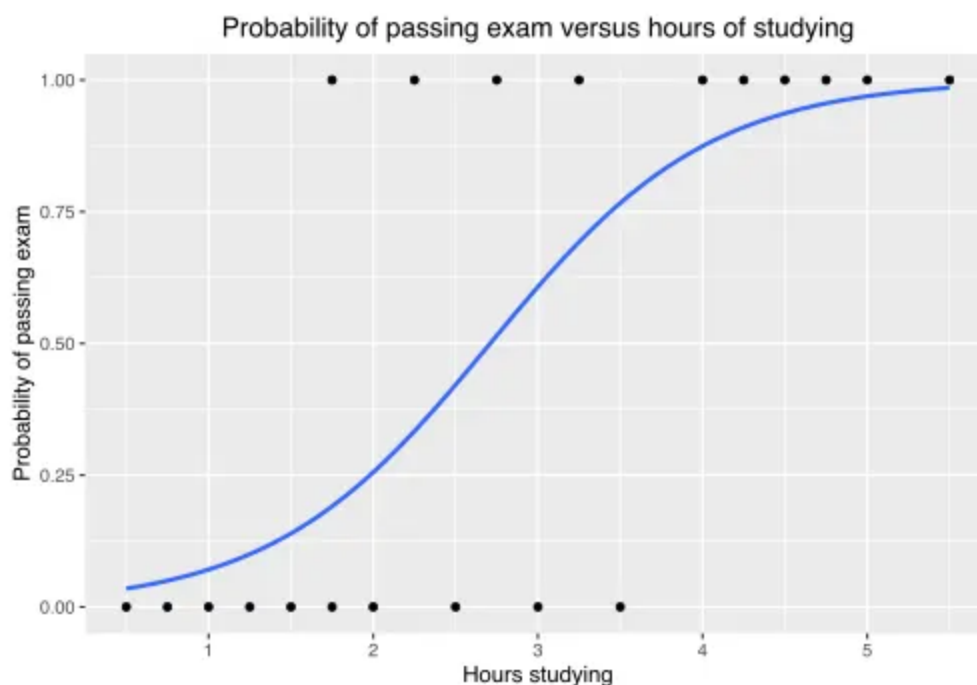
That single connection unlocks a lot. Let me name the famous ones.

The humble average is maximum likelihood under a bell curve, as we just saw. You have been doing it for years.

Logistic regression is the workhorse for yes or no questions. Will this customer cancel. Is this email junk. Does this patient look high risk given their age and blood pressure. Those are coin flips where the coin's bias changes with the situation. Logistic regression learns how the situation tips the coin, and it learns it by maximizing the likelihood of every yes and no it has ever seen. It is our coin example after it grew up, put on a badge, and got hired by a hospital.

Your phone keyboard guesses the next word that makes your sentence so far the least surprising, based on mountains of text it studied. That is likelihood reasoning wearing a hoodie.

And huge stretches of machine learning train by writing down “how likely is the training data under the current settings,” then nudging millions of internal knobs, a hair at a time, to push that likelihood up. “Training a model” and “climbing a likelihood hill” are very often the exact same sentence. The hill just has millions of dimensions, and the computer has far more patience than I do on a Sunday.



The S curve is just a coin whose chance of heads slides as the situation changes.



One little idea. A ridiculous number of day jobs.

[Open in app](#)

≡ Medium

🔍 Search

✍ Write



The honest part, where the coin lies to your face

I would be a bad guide if I left you thinking this method is a flawless oracle. It is powerful and it is everywhere and it has two genuinely embarrassing weak spots. Meet them now so they never ambush you later.

Small data lies, and maximum likelihood believes it instantly. Flip the coin twice. Get two heads. With a totally straight face, the method announces that the coin lands heads 100 percent of the time. Two out of two, so p equals one. No doubt. No shame. It looked at two flips and declared tails physically impossible.

That is absurd, and it reveals something real. Maximum likelihood gives you the best explanation for the data you happened to get. It does not, on its own, tell you how much to trust that explanation. Two flips, trust it about as far as you can spit. Ten thousand flips, trust it a lot. The estimate is only as honest as the data feeding it, and on thin data it will say wild things with the calm confidence of a man who has never once been wrong and never will admit it.

It also falls in love with your data, which we call overfitting. Because the method chases the best possible fit to the data it has seen, a flexible enough model can end up memorizing the lumps and noise of your specific dataset instead of the real pattern. It aces the practice test and bombs the real exam, like a student who memorized the answer key and learned nothing. Real practitioners spend a lot of energy keeping this thing on a short leash.

There is even a quiet third one worth a nod. For the bell curve, the maximum likelihood spread is slightly too small on average, a tiny built in lowball known as bias. It is small, it fades as data grows, and there is a famous one line fix, but it is a neat reminder that “best fit to this data” and “true value” are cousins, not twins.

None of this means the idea is broken. It means it is a tool, not a wizard. Knowing exactly where a tool fails is the whole difference between using it and being used by it.



With two flips it will swear the coin is magic. Confidence is free. Data is not.

Back to the friend and the suspicious coin

My friend is still across the table. The coin is still sitting on its little pile of seven heads and three tails. The leaf blower next door has finally stopped.

I can now tell him the honest truth. Our single best guess is that this coin lands heads about 70 percent of the time, because that is the explanation that makes what we saw the least surprising. But ten flips is almost nothing. Do not bet the rent. Flip it two hundred more times and ask me again.

That whole answer is maximum likelihood estimation. The best explanation half, and the but consider how much data you have half. Hold both halves and you understand it better than any formula could teach you alone.

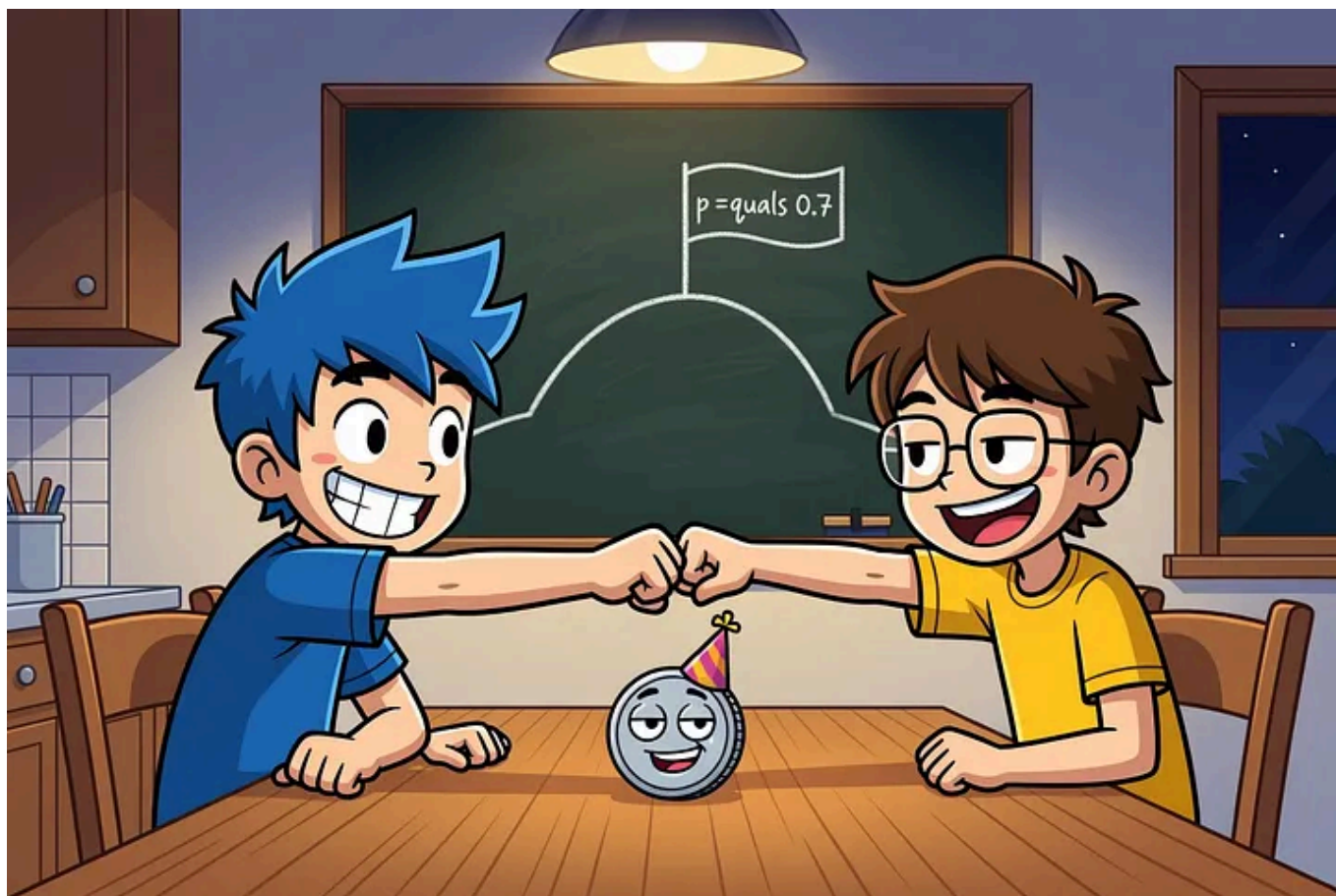
So, one more time, the sentence we started with.

Maximum likelihood estimation means picking the explanation that makes your data look the least surprising.

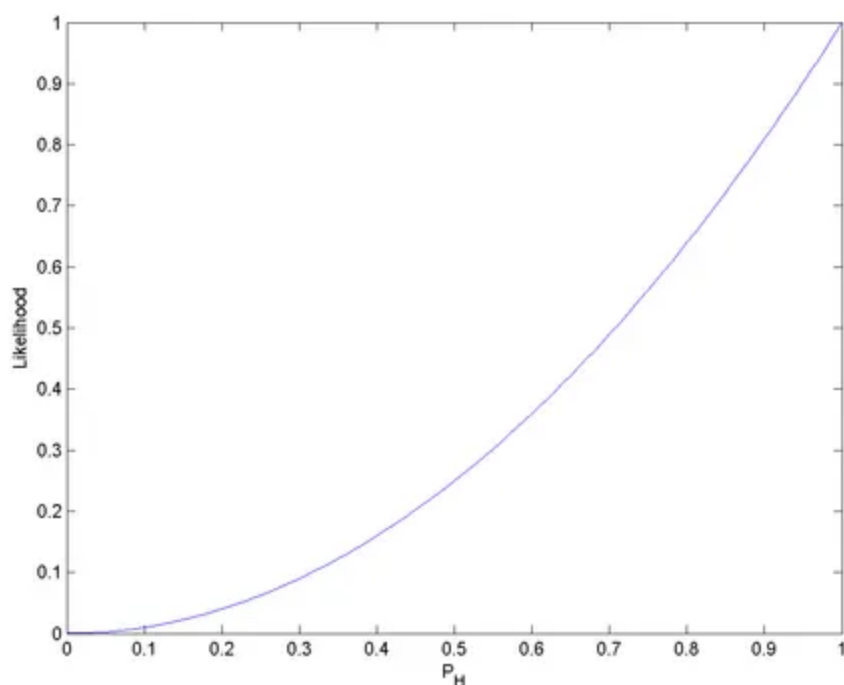
You now know what explanation means. A setting of the hidden knobs. You know what least surprising means. The top of the likelihood hill. You met the log trick that keeps the math from collapsing, the four step recipe that works on anything, the Python that finds the peak, the century old story of where it came from, the bell curve where it becomes the plain average, the empire of tech it secretly runs, and the two ways it will lie to your face.

That is not a vibe of the topic. That is the topic, front to back. Not bad for an idea you reinvented by accident in the first thirty seconds, with a barking dog and a sticky Enter key.

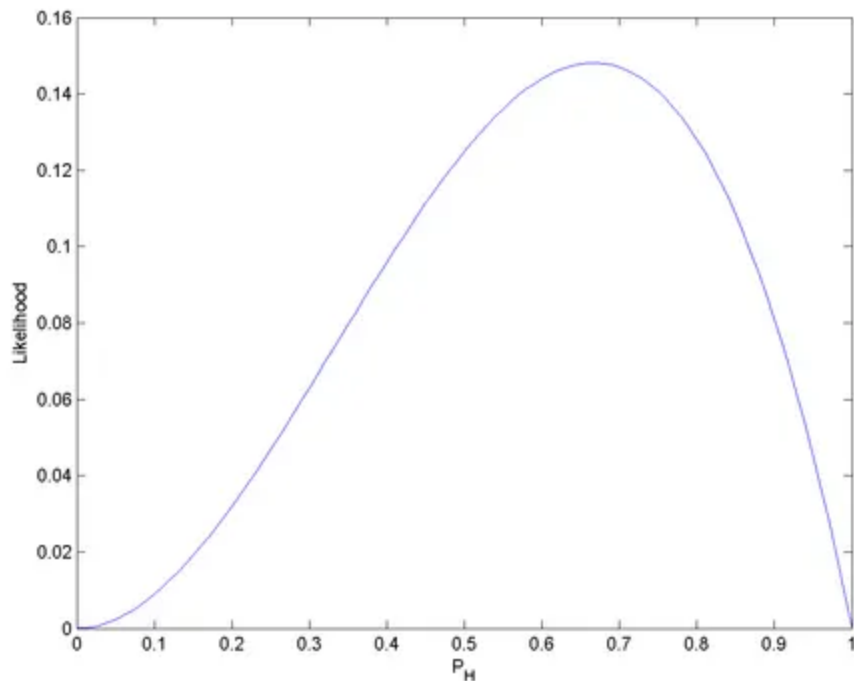
The dial was hidden the whole time. You just learned how to find where it most likely sits. And almost everything else in statistics, every confidence interval, every test, every model you will ever train, is some clever remix of that one move. The world hides its knobs. We reason, carefully and a little stubbornly, back toward where they probably sit.



Case closed. The coin is a little rigged and you are a little bit a statistician now.



The likelihood function (p_H^2) for the probability of a coin landing heads-up (without prior knowledge of the coin's fairness), given that we have observed HH.



The likelihood function $(p_H)^2(1-p_H)$ for the probability of a coin landing heads-up (without prior knowledge of the coin's fairness), given that we have observed HHT.

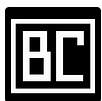
Data Science ✓

Machine Learning ✓

Statistics +

Python +

Artificial Intelligence ✓



Published in Bootcamp

113K followers · Last published 2 hours ago

[Follow](#)

From idea to product, one lesson at a time. To submit your story:
<https://tinyurl.com/bootspub1>



Written by Kamrun Nahar

1.3K followers · 21 following

[Follow](#)

I write about machine learning, deep learning, and statistics with Python, making complex ideas ridiculously clear.
